Does Your MCP Server Actually Follow the Protocol? A Large-Scale Execution-Based Conformance Study of the Model Context Protocol Ecosystem

Ahmed Faraj
Suez Canal University
ahmed.faraj.cs@gmail.com

Abstract

The Model Context Protocol (MCP) has become the de-facto standard for connecting LLM agents to external tools, and public registries now list tens of thousands of servers. Yet every prior empirical study of this ecosystem is *static*: scanning source code, mining issue trackers, or reading registry metadata. Nobody has *executed* the ecosystem at scale to ask a prior question—do these servers actually run, and do they actually follow the protocol? We present the first execution-based conformance study of the MCP server ecosystem. From the official registry (17,432 unique servers) we draw a random sample of 279 self-contained, locally-installable servers, launch each in a network-isolated sandbox, and drive it through a conformance harness that exercises the initialization lifecycle, tool listing, declared-schema validity, and behavior under three inputs a real agent session produces: unknown tool names, wrong-typed arguments, and malformed protocol frames. We find that only 57.3% (95% CI 51.5–63.0) of installable servers successfully start and complete a handshake; that 88.1% (95% CI 82.2–92.3) of responding servers violate the specification's error-reporting requirement in an identical way traceable to SDK defaults; and that a non-trivial minority silently accept ill-typed tool arguments or fail on malformed input. We release the harness and dataset as reusable infrastructure. Our results show that ecosystem-scale protocol conformance—not just security—is an unmet need, and that the gap between the written specification and deployed practice is wide and systematic.

1 Introduction

[Section drafted after final data.]

2 Background and Related Work

2.1 The Model Context Protocol

MCP standardizes how LLM agents discover and invoke external tools. A client and server negotiate a protocol version and capabilities during an `initialize` handshake, after which the client can list tools (`tools/list`) and invoke them (`tools/call`). Messages follow JSON-RPC 2.0, including its error-object semantics and reserved error codes. The most common deployment is a local server communicating over stdio, where stdout carries protocol traffic exclusively. The specification is versioned by date, and servers may negotiate down to a version they support.

2.2 Static studies of the MCP ecosystem

The ecosystem has been studied at scale, but statically. Health-and-maintainability work analyzes source and metadata of roughly 1,900 servers with static scanners and code-smell detectors [**?**]. Registry-measurement

work crawls multiple marketplaces and characterizes scale, maintenance, and supply-chain concentration for on the order of 8,000 servers, but from listing metadata rather than execution [**?**]. Fault taxonomies derive categories of runtime misbehavior by *mining GitHub issues* [**?**, **?**]; these report that tool faults and schema-enforcement problems dominate the fault landscape—smoke that our harness measures directly as fire. None of these works executes servers to test conformance.

2.3 Execution-based work is security-focused

Where prior work does execute MCP servers, it targets security rather than conformance, and on small or synthetic populations. Sandboxed behavioral scanners and taint-style analyzers hunt vulnerabilities on curated or intentionally vulnerable server sets [**?**, **?**]; a security-invariants benchmark *deliberately avoids live third-party execution*, using fixtures and a 40-repository metadata sample [**?**]; malicious-server detection and authentication studies likewise scope to security surfaces [**?**, **?**]. Protocol conformance—whether a server that is neither malicious nor vulnerable actually behaves as the specification requires—is a distinct question that this line of work does not address.

2.4 Conformance tooling and the gap we fill

The protocol maintainers ship an official conformance suite, but it tests the handful of official SDKs in continuous integration; it has not been pointed at the deployed ecosystem. The community has begun proposing a pre-publish conformance checklist for server authors, evidencing demand for exactly the measurement we provide. Practitioner audits have reported striking headline numbers (e.g., that a majority of listed servers are unmaintained), but without published methodology, data, or protocol-level testing. To our knowledge, ours is the first reproducible, protocol-level, execution-based conformance measurement of the MCP server ecosystem, released with an open harness and dataset. In spirit it follows a long line of empirical software-engineering measurement studies that execute a population to characterize how it really behaves, rather than how its documentation says it should.

3 Methodology

We measure the MCP server ecosystem by *executing* it. This section defines the sampling frame, the conformance harness, the sandbox, and the grading rules. The harness and the released dataset make every number reproducible.

3.1 Sampling frame

We crawl the official MCP registry (`registry.modelcontextprotocol.io`) via its cursor-paginated API, retaining the latest published version of each server. This yields 17,432 unique servers. Each server declares zero or more *packages* (installable artifacts on npm, PyPI, OCI, NuGet, or Cargo) and/or *remotes* (hosted HTTP endpoints). Our study population is the set of servers that are (i) locally installable, (ii) use the `stdio` transport, (iii) distributed on npm or PyPI (the two dominant registries), and (iv) *self-contained*: they declare no required environment variable or argument that lacks a default. Criterion (iv) restricts us to servers that a client could launch without provisioning secrets, which both bounds the ethics of mass execution and defines a clean, reproducible population. We report the exclusion of remote-only and credential-requiring servers explicitly (Section **??**) and treat the resulting frame as the denominator for all rates. From this frame we draw a fixed pseudo-random sample (seeded) of 279 servers.

3.2 Conformance harness

The harness, `mcpprobe`, is a deliberately minimal, hand-written JSON-RPC client speaking line-delimited messages over the server's stdio. We do *not* use an official SDK client: to observe raw behavior—including responses to malformed frames—the probe must not inherit any SDK-side correction or validation. The harness offers protocol version `2025-06-18` at initialization, records the version the server *negotiates*, and grades all subsequent checks against that negotiated version rather than the latest specification.

Each server is driven through the following checks, whose identifiers align with the categories of the official `modelcontextprotocol/conformance` suite where an analogue exists:

- **server-initialize**: the `initialize` request must return a result carrying a protocol version and a capabilities object; the harness then sends `notifications/initialized`.
- **ping**: a `ping` must return an empty-object result.
- **tools-list**: if the server advertises the tools capability, `tools/list` must return an array in which every entry has a `name` and an `inputSchema` object.
- **tools-schema-valid**: each declared `inputSchema` must be a valid JSON Schema (checked by meta-validation).
- **tools-call-unknown**: calling a non-existent tool must be rejected. The specification requires a JSON-RPC error (`-32602`/`-32601`).
- **tools-call-invalid-args**: for a tool with a typed required property, calling it with a wrong-typed value must be rejected (protocol error or an `isError` result).
- **malformed-json**: after receiving a syntactically invalid frame, the server must not crash or hang; a subsequent `ping` must still succeed.
- **stdout-purity**: under the stdio transport, the server must emit only protocol messages on stdout; any non-JSON line is a channel-corruption defect.

3.3 Grading and the error-as-result category

Verdicts are pass, fail, warn, skip, and one category introduced by calibration: error-as-result. When validating the harness against the official reference servers (`server-memory`, `server-everything`), we found they answer an unknown-tool call not with the specified JSON-RPC `-32602` error but with a *successful* response whose `result.isError` flag is set. Because this behavior originates in the official SDKs and is therefore ecosystem-wide, we grade it as its own category rather than a failure, and quantify its prevalence as a first-class result: it is the clearest instance of specification text diverging from universal practice.

3.4 Sandbox

Executing thousands of untrusted packages demands isolation. Each server runs in a fresh container (`node:22-slim` for npm, an `astral-sh/uv` image for PyPI) with a memory cap, a single CPU, a PID limit, `no-new-privileges`, and no host filesystem mounts other than a package-cache volume. Measurement uses a **two-phase** protocol: an online install pass populates the cache, then the conformance probe runs with `--network=none`. No server code has network access while it is being measured, which both hardens the experiment and removes network flakiness and server-side outbound calls as confounders.

3.5 Funnel accounting

Because "does it even run" is itself a research question, we count every stage: declared → eligible (self-contained stdio npm/PyPI) → sampled → container started → process launched → handshake completed → probed. Servers that never reach a handshake are classified into startup-failure categories (Section **??**) from their exit code and stderr, so that the yield loss is characterized rather than merely reported.

4 Results

[Section drafted after final data.]

5 Discussion

[Section drafted after final data.]

6 Threats to Validity

[Section drafted after final data.]

7 Ethics and Responsible Disclosure

[Section drafted after final data.]

8 Conclusion

[Section drafted after final data.]